

PROBLEM STATEMENT

I am framing the assignment inside a specific business context, because it is only safe to switch into engineering mode once you understand that context — who is paying for the model usage, who all the stakeholders are, and where the real pain points sit.

For the sake of efficient communication, let me frame it as a story.

Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists.

Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI system, and every call costs money.

I am framing this assignment as an experimentation phase before sitting down with Isaac, so that when we talk, I can discuss cost in concrete terms.

My goal going in is to already know the levers: which shortcuts make verification cheaper, and how much accuracy each one costs

Then when a grader tells me one step is critical, I can weigh it on the spot — spend more and give that step its own dedicated AI check, or save money where accuracy matters less.

Because I will have already measured the cost and the quality of some of the shortcuts.

Let's get to precisely defining what we're solving for.

Concretely, the problem I'm solving is this:

Given the workload of verifying IPhO-level physics solutions, design an inference-time execution system that cuts cost-per-verification by 50% versus a strong single-frontier-model baseline, while catching bad solutions at least as reliably as that baseline.

This is a faithful reading of the assignment, as the brief asks for an "AI execution system" that reduces "inference costs" — and its own vocabulary (workload, application, model family, infrastructure layer, combination of techniques) signals that "inference" is meant broadly: the cost of running a pipeline to produce an output, not one model's forward pass. So I read inference cost as dollars per solution verified, summed across every model and tool call the system makes.

The statement also pins down the two things a percentage claim needs to be credible. It fixes a reference point — a strong single-frontier-model baseline, the kind a competent engineer would actually ship— and it treats "equal quality" as a floor rather than a goal: the cheap system has to catch bad solutions at least as reliably as that baseline, so any savings I report are savings at held-constant quality. What it deliberately leaves open is how. It names no cascade, no router, no caching scheme, because the assignment is open-ended

on technique and the honest version of this work lets the evidence decide which levers pay off. That keeps the definition of the problem separable from the solution to it — which, given the story above, is the whole point: **I want to walk into the conversation with Isaac already knowing what each shortcut costs and what it buys.**

NOTES

- The verifier system isn't necessarily a realtime system, it's a system you run periodically, so whenever talking to LLM should use Batch API to send multiple reqs together instead of one -> this alone yields 50% cost savings w/ certain models